Introduction to NSubstitute

In this example, we use a mocking library like **NSubstitute**.
NSubstitute allows us to create a fake version of a dependency, known as a *substitute*. Instead of calling the real implementation, we define how the substitute should behave during the test. This gives us full control over inputs and outputs without needing a real database or data source.

In this example, we are testing a UserService that depends on an IUserRepository.

Using NSubstitute, we:

- Create a substitute for IUserRepository
- Configure return values for specific inputs (e.g. returning a user for ID = 1)
- Handle edge cases such as invalid IDs or missing users
- Simulate exceptions for invalid scenarios
- Verify that methods are called as expected

You will also see more advanced usage, such as:

- Argument matching with Arg.Is and Arg.Any
- Returning dynamic results using a delegate
- Throwing exceptions with .Throws()
- Verifying interactions using .Received()

```
using System;
using Microsoft.VisualStudio.TestTools.UnitTesting;
using NSubstitute;
using NSubstitute.ExceptionExtensions;
using System.Collections.Generic;

[TestClass]
public class UserServiceTests
{
    [TestInitialize]
    public void setup()
    {
        var repo = Substitute.For<IUserRepository>();

        repo.GetById(1).Returns(new User(1, "Mike"));
        repo.GetById(Arg.Is<int>(id => id > 100)).Returns((User)null);
        repo.GetById(Arg.Is<int>(id => id < 0)).Throws(new
                                                ArgumentOutOfRangeException());

        setupManyUsersDemo(repo);
    }

    private void setupManyUsersDemo(IUserRepository repo)
    {
        var users = new Dictionary<int, User>
        {
            { 1, new User(1, "Mike" ) },
            { 2, new User(2, "Anna" ) },
            { 3, new User(3, "John") }
        };

        repo.GetById(Arg.Any<int>()).Returns(call =>
        {
            int id = call.Arg<int>();
            return users.ContainsKey(id) ? users[id] : null;
        });
    }


    [TestMethod]
    public void GetUserName_returns_name_when_user_exists()
    {
        // Arrange
        var repo = Substitute.For<IUserRepository>();

        repo.GetById(1).Returns(new User(1, "Mike"));

        var service = new UserService(repo);

        // Act
        var result = service.GetUserName(1);

        // Assert
        Assert.AreEqual("Mike", result);
        repo.Received(1).GetById(1);
    }
```

```
    [TestMethod]
    public void GetUserName_returns_null_when_user_not_found()
    {
        // Arrange
        var repo = Substitute.For<IUserRepository>();

        repo.GetById(Arg.Is<int>(id => id > 100)).Returns((User)null);

        var service = new UserService(repo);

        // Act
        var result = service.GetUserName(99);

        // Assert
        Assert.IsNull(result);
        repo.Received(1).GetById(99);
    }

    [TestMethod]
    [ExpectedException(typeof(ArgumentOutOfRangeException))]
    public void GetUserName_Throws_Exception_When_ID_more_than_100()
    {
        // Arrange
        var repo = Substitute.For<IUserRepository>();

        repo.GetById(Arg.Is<int>(id => id < 0)).Throws(new
                                        ArgumentOutOfRangeException());

        var service = new UserService(repo);

        // Act
        service.GetUserName(-1);
    }
}
```

Classes under test

```
public class User
{
    public User(int id, string name)
    {
        Id = id;
        Name = name;
    }

    public int Id { get; set; }
    public string Name { get; set; }

}

public interface IUserRepository
{
    User GetById(int id);
}

public class UserService
{
    private readonly IUserRepository _repository;

    public UserService(IUserRepository repository)
    {
        _repository = repository;
    }

    public string GetUserName(int userId)
    {
        var user = _repository.GetById(userId);

        if (user == null)
        {
            return null;
        }

        return user.Name;
    }
}
```